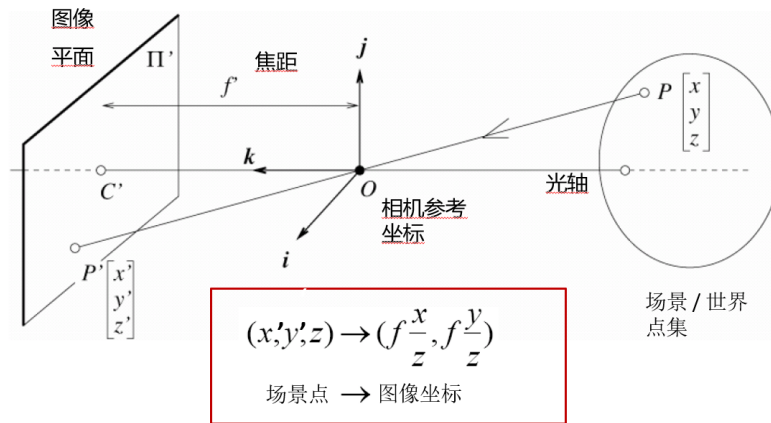


计算机视觉

- 复习计划
 - 看总结PPT题型
 - 对照着一边看一边记在下展开
- 第零篇：绪论
 - 什么是计算机视觉？
 - 计算机视觉是一门研究如何使机器“看”的科学，更进一步的说，就是指用摄影机和电脑代替人眼对目标进行识别、跟踪和测量等机器视觉，并进一步做图形处理，使电脑处理成为更适合人眼观察或传送给仪器检测的图像。
 - 作为一个科学学科，计算机视觉研究相关的理论和技术，试图建立能够从图像或者多维数据中获取‘信息’的人工智能系统。
 - 计算机视觉的主要应用
 - 3D建模
 - 地图映射
 - 计算摄影
 - 手写字符识别
 - 人脸检测
 - 物体识别
 - 生物视觉识别
 - 目标识别
 - 动画3D建模
 - AR/VR
 - 自动驾驶
 - 全景拼接
 - 机器自主导航
 - 工业视觉
 - ...
- 第一篇：图像分析（50%）
 - 1.1 图像形成√
 - 针孔模型
 - world->pix

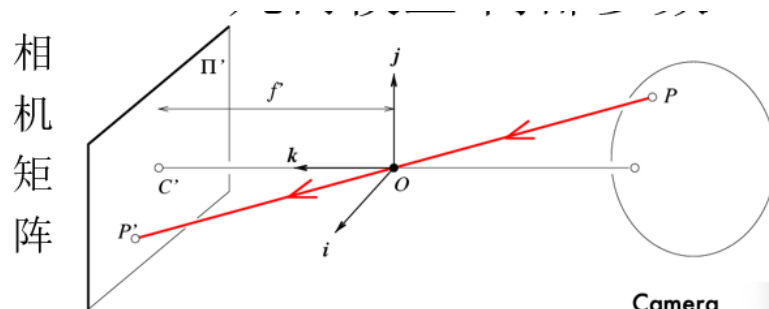


orsyth and Ponce

孔径扩大 $\rightarrow$ 1个像素由多个场景点发出的光贡献 $\rightarrow$ 越模糊

• (重点) 几何模型

- K矩阵：注意单位



$$P' = M P$$

$$= K \begin{bmatrix} I & 0 \end{bmatrix} P$$

$$P' = \begin{bmatrix} \alpha & 0 & c_x & 0 \\ 0 & \beta & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Camera matrix K

$(x, y, z) \rightarrow (f k \frac{x}{z} + c_x, f l \frac{y}{z} + c_y)$

α β $[Eq. 6]$

Units: k, l : pixel/m f : m Non-square pixels α, β : pixel

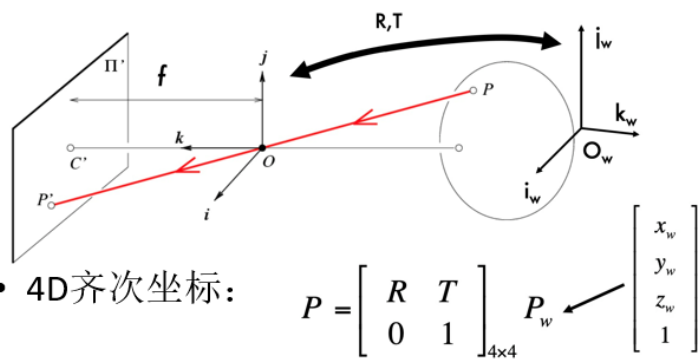
$C' = [c_x, c_y]$

θ

$C = [c_x, c_y]$

$$P' = \begin{bmatrix} \alpha & -\alpha \cot \theta & c_x & 0 \\ 0 & \frac{\beta}{\sin \theta} & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- RT



- 4D齐次坐标:

$$P = \begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix}_{4 \times 4} P_w \rightarrow \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

$$P' = K \begin{bmatrix} I & 0 \end{bmatrix} P = K \begin{bmatrix} I & 0 \end{bmatrix} \begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix}_{4 \times 4} P_w = K \begin{bmatrix} R & T \end{bmatrix} P_w$$

Internal parameters External parameters

M [Eq.11]

参数个数: 5 (K) + 3 (R) + 3 (T) = 11

- 投射方程基本矩阵表示

- 透视投影基本方程矩阵表示:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f' & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z/f' \end{bmatrix} \Rightarrow (f' \frac{x}{z}, f' \frac{y}{z})$$

齐次坐标除以z坐标
转换成非齐次坐标

添加一维坐标转换为齐次坐标

f'表示焦距, z表示物距

弱透视投影 $f' \ll z$

正交投影 f' 很大

• 1.3 滤波和边缘检测

- 噪声类型

椒盐噪声:

随机的黑白像素点

脉冲噪声:

随机的白点

高斯噪声:

均匀区域噪声服从

高斯分布

- 滤波和卷积

- 相关和卷积

卷积

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H \star F$$

相关

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

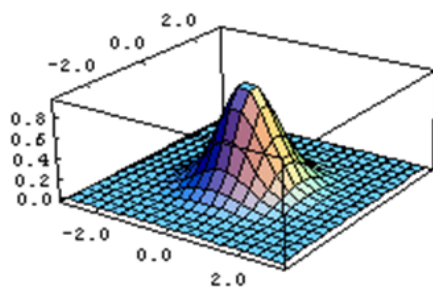
- padding: 一般当核为 $(2k+1) \times (2k+1)$ 时padding k个单位
- 不考虑翻转
- 滤波类型

中值滤波: 适用于脉冲、椒盐噪声->中位值替代中间位置

- 高斯滤波器

内核近似为
高斯函数:

$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{\sigma^2}}$$



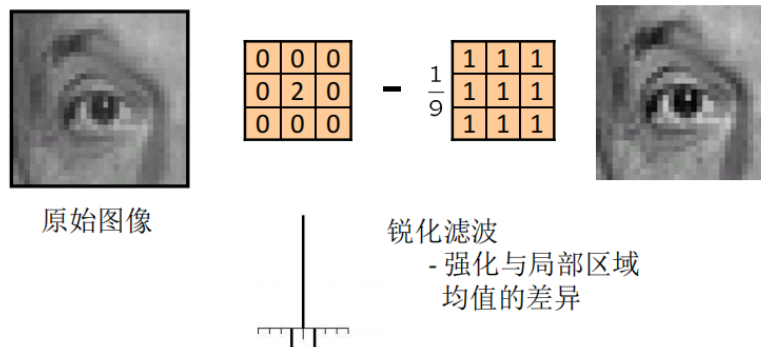
- 参数设置与效果

sigma 方差 越大 图像越平滑越模糊

kernel_size 越大越模糊

●

- 卷积核组合运算的意义:



- 模板匹配

- 思路与原理

滤波器可以近似 看作模板匹配。

前面被当作图像去噪工具，现在考虑滤波如何实现特征提取/实现原始像素到中间表达的映射从而用于后续处理目标：从数据中去除冗余，保留有用成分原理：使用归一化相关值在场景中寻找给定的模板图案/归一化的作用是控制相关值的绝对大小特征：如果尺度、方向和表面形状基本相似，匹配是有意义的

- 求最大响应

卷积（相关）运算是在计算每个位置与该模式的相似程度，或者说每个位置具有该模式的分量有多少，当前位置与该模式越像，响应越强。

- 边缘检测

- 边缘的定义：图像中亮度（或者其他属性）发生快速变化的区域

- 可供参考：[\(49条消息\) 6.边缘检测：梯度——有限差分、图像的偏导数、离散梯度 3 Tom Boom的博客-CSDN博客 离散梯度算子](#)

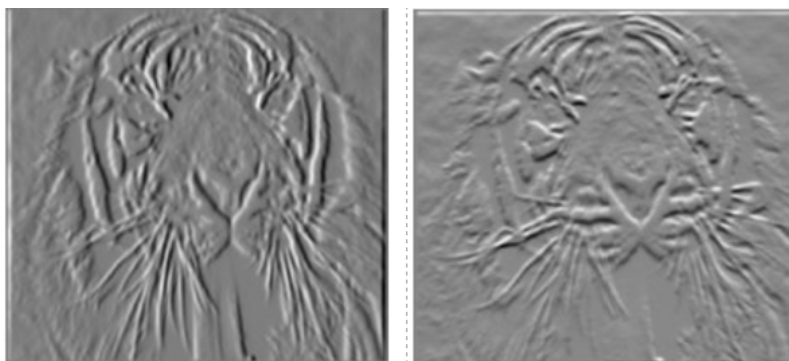
- 步骤：平滑-边缘增强：检测子滤波-边缘定位

-

- 判断方向

- 边缘：图像强度函数剧烈变化的位置

-



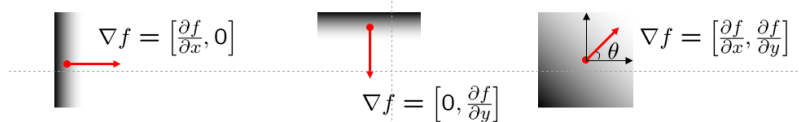
$$\frac{\partial f(x, y)}{\partial x} \quad \frac{\partial f(x, y)}{\partial y}$$

- 图像梯度

图像的梯度:

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

梯度方向为强度变化最剧烈的方向



边缘方向为梯度方向，可以表示为:

$$\theta = \tan^{-1} \left(\frac{\partial f / \partial y}{\partial f / \partial x} \right)$$

边缘强度为梯度幅值:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

- 不同检测子类型

- 参考文章: [\(49条消息\) 常见边缘检测对比 \(Roberts算子、Prewitt算子、Sobel算子、Laplacian算子、Canny算子\) 荣仔! 最靓的仔! 的博客-CSDN博客 sobelprewittrobertscanny的区别](#)

- prewitt sobel roberts

Prewitt: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$

Sobel: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$

Roberts: $M_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

- 高斯拉普拉斯检测子

- 高斯函数二次偏导求和

$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}} \quad \frac{\partial}{\partial x} h_{\sigma}(u, v) \quad \nabla^2 h_{\sigma}(u, v)$$

- ∇^2 为laplace检测子:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

用GOL算子卷积的边缘在过零点

- canny

- 使用高斯检测子计算
- 确定梯度幅值和方向
- 非最大化抑制:
多像素宽度窄带边缘细化为单像素宽度
- 边缘阈值和连接 (滞后法):
 - 定义双阈值: 低和高
 - 高阈值确定边缘起始, 低阈值进行边缘连接

非最大化抑制: 多像素宽度带边缘细化为单像素宽度。

双阈值: >上界: 保留 <下界: 舍弃 之间: 连接处使用

- 性能评估

精确率: 检测结果正向中正确像素的比率

$$P = \frac{tp}{tp + fp}$$

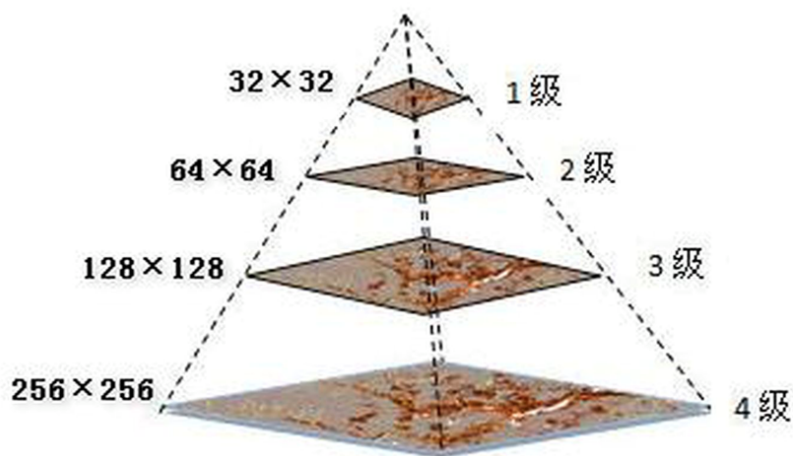
召回率: 标签为边缘像素中被正确检测的比率

$$R = \frac{tp}{tp + fn}$$

	标签正向	标签负向
检测结果正向	tp	fp
检测结果负向	fn	tn

- 1.4 多尺度空间

- Gauss金字塔



对原图像用高斯滤波进行模糊，每一层都要进行。进行间隔为2的下采样。

• 1.5 局部特征提取

• Harris角点检测

• 原理

角点定义：沿着任何方向移动都有显著变化的点。

• 步骤

– 设定窗口，计算矩阵 M 得到整个图像的响应值 R

– 找出具有较大响应值得点 ($R > \text{threshold}$)

– 确定 R 局部极值

R 局部极值 近似于非极大值抑制 筛选出梯度最大的值

具有旋转不变性，不具有尺度不变性

• 局部区域描述

点描述子：不变性和可鉴别的

sift： $4 \times 4 \times 8 = 128$ 维度 描述某区域的十六个单元的特征

旋转不变性：关键点梯度主方向

• 特征匹配

1.特征点提取

2.特征点匹配

3.匹配特征点筛选

4.变换参数求解

• SIFT

• 原理

1) 构建多尺度空间

进行高斯平滑+下采样，得到高斯金字塔

2) DoG找关键点

对高斯金字塔的相邻图像做差分图得到DOG金字塔。对于DoG中的像素与同尺度8个相邻点+上下相邻尺度对应的9*2个点进行比较，得到尺度空间和二维图像空间的极值点。利用DoG曲率去除边缘的特征点，最终得到真正的关键点。

3) 关键点方向匹配

采用梯度直方图统计法，梯度直方图中分0-360度，统计的就是图像局部区域的梯度分布情况，选取直方图的峰值作为关键点的主方向；同时保留峰值大于主方向峰值80%的方向作为关键点的辅方向。

4) 关键点描述子

每一个关键点，都有位置、尺度盒和方向三个信息，需要为之建立一个描述子，包含其周围对其有贡献的像素点，这里一共取 $4*4*8=128$ 维度的向量作为描述子表示。

5) 关键点匹配

匹配两个图中的关键点，便于进一步操作。此时有多种方法，如穷举法暴力匹配，Kd树等（K临近的实现方法之一），原理都是计算描述子之间的欧式距离，选取最近的作为一对匹配。

● 步骤

- Gauss金字塔
- LOG
- DOG极值点
- 极值点筛选
- 计算特征描述子
- ...等

● 效果

- 尺度旋转不变性
- 光照不变性

● 1.6 机器学习与图像分类

- 1.收集图像和对应标签的数据集
- 2.使用机器学习去训练分类器
- 3.使用新图像验证分类器

监督学习的目的：对于输入数据提供一个最佳预测输出

● K临近算法

● 原理

最近：图像比较距离测度 l_1/l_2 ，找出距离最小的训练图像返回其对应标签。测试复杂度高且缓慢，正确率不高38.6%左右。重点在于距离度量和K值确定，K值取7左右为最佳。

● 优缺点

优点

- 简单好用，容易理解，精度高，理论成熟，既可以用来做分类也可以用来做回归；
- 可用于数值型数据和离散型数据；
- 训练时间复杂度为 $O(n)$ ；无数据输入假定；
- 对异常值不敏感。

缺点：

- 计算复杂性高；空间复杂性高；
- 样本不平衡问题（即有些类别的样本数量很多，而其它样本的数量很少）；
- 一般数值很大的时候不用这个，计算量太大。但是单个样本又不能太少，否则容易发生误分。
- 最大的缺点是无法给出数据的内在含义。

• 线性分类器

- 式子
- 三个角度理解
 - 几何
 - 代数 $f(x, W) = Wx + b$
 - 可视化

• 优缺点

- 算法简单和具有“学习”能力。线性分类器速度快、编程方便，但是可能拟合效果不会很好。

• 损失函数

预判正确----->损失较低 预判错误----->损失较大

反向传播去更正参数

- 掌握算法
 - 交叉熵

模型1:

预测	真实	是否正确
0.3 0.3 0.4	0 0 1 (猪)	正确
0.3 0.4 0.3	0 1 0 (狗)	正确
0.1 0.2 0.7	1 0 0 (猫)	错误

模型1:

$$\begin{aligned}\text{sample 1 loss} &= -(0 \times \log 0.3 + 0 \times \log 0.3 + 1 \times \log 0.4) = 0.91 \\ \text{sample 2 loss} &= -(0 \times \log 0.3 + 1 \times \log 0.4 + 0 \times \log 0.3) = 0.91 \\ \text{sample 3 loss} &= -(1 \times \log 0.1 + 0 \times \log 0.2 + 0 \times \log 0.7) = 2.30\end{aligned}$$

对所有样本的loss求平均:

$$L = \frac{0.91 + 0.91 + 2.3}{3} = 1.37$$

交叉熵损失(多元Logistic回归)

将分类器输出的分值得解释为概率:



SVM

	cat	car	frog
cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Loss	2.9	0	12.9

给定一副图像 (x_i, y_i)
(x_i 为图像, y_i 为标签)

令 $s = f(x_i, W)$ 为响应分值

则SVM 损失定义为:

$$\begin{aligned}L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \max(0, 2.2 - (-3.1) + 1) \\ &\quad + \max(0, 2.5 - (-3.1) + 1) \\ &= \max(0, 6.3) + \max(0, 6.6) \\ &= 6.3 + 6.6 \\ &= 12.9\end{aligned}$$

最优化

- 梯度下降
- SGD、BGD、MBGD

SGD 随机选择一批神经元 计算梯度下降 /N

- Momentum

1.7 神经网络

- 反向传播计算

后向传播算法:
举例

$$f(x, y, z) = (x + y)z$$

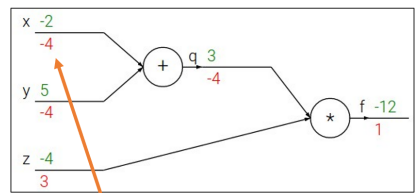
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



链式法则

$$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \frac{\partial f}{\partial q}$$

$$\frac{\partial q}{\partial x} = 1$$

下游梯度

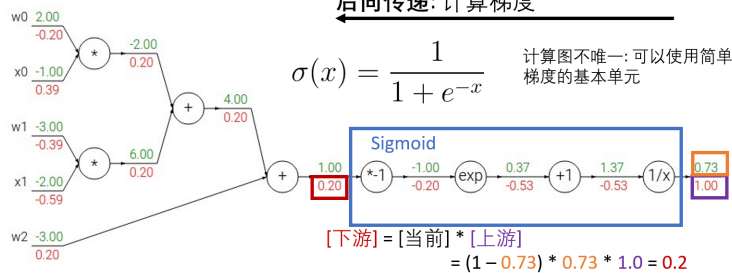
局部梯度

上游梯度

另一个示例

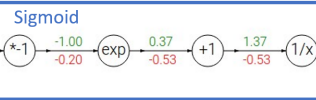
$$f(x, w) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}} = \sigma(w_0 x_0 + w_1 x_1 + w_2)$$

后向传递: 计算梯度



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

计算图不唯一: 可以使用简单梯度的基本单元



$$[\text{下游}] = [\text{当前}] * [\text{上游}] = (1 - 0.73) * 0.73 * 1.0 = 0.2$$

Sigmoid local gradient: $\frac{\partial}{\partial x} [\sigma(x)] = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$

卷积神经网络

卷积

卷积总结

输入: $C_{in} \times H \times W$

超参数:

- 核大小: $K_H \times K_W$
- 核数量: C_{out}
- 填充: P
- 步长: S

权重矩阵: $C_{out} \times C_{in} \times K_H \times K_W$ 个
大小为 $C_{in} \times K_H \times K_W$ 滤波核

偏置向量: C_{out}

输出大小: $C_{out} \times H' \times W'$ 这里:

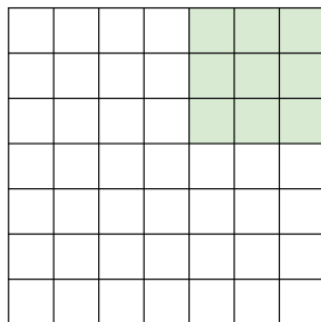
- $H' = (H - K + 2P) / S + 1$
- $W' = (W - K + 2P) / S + 1$

常用设置:

- $K_H = K_W$ (等尺寸滤波核)
- $P = (K - 1) / 2$ ("相同尺寸" 填充)
- $C_{in}, C_{out} = 32, 64, 128, 256$ (2的幂次方)
- $K = 3, P = 1, S = 1$ (3x3 conv)
- $K = 5, P = 2, S = 1$ (5x5 conv)
- $K = 1, P = 0, S = 1$ (1x1 conv)
- $K = 3, P = 1, S = 2$ (2倍下采样)

输入输出尺寸计算

卷积步长



输入: 7x7
滤波: 3x3 输出: 3x3
步长: 2

通常:
输入: W
滤波: K
填充: P
步长: S
输出: $(W - K + 2P) / S + 1$

- 参数量, 乘法次数

输入: 3 x 32 x 32
10 5x5 滤波组步长1, 填充2



输出大小: 10 x 32 x 32

学习参数规模: 760

乘法次数: 768,000

$10 * 32 * 32 = 10,240$ 输出; 每个输出是两个 3x5x5 张量 (75 个元素) 的点积; 总计 = $75 * 10240 = 768K$

输入: 3 x 32 x 32

10 5x5 滤波组步长1, 填充2

输出大小: 10 x 32 x 32

学习参数规模: 760

每个滤波核参数: $3 * 5 * 5 + 1$ (偏置) = 76

10 个滤波核, 所以总共 $10 * 76 = 760$

- 池化

输入: C x H x W

超参数:

- 内核尺寸: K
- 步长: S
- 池化函数(max, avg)

输出: C x H' x W' 这里

- $H' = (H - K) / S + 1$
- $W' = (W - K) / S + 1$

可学习的参数: None!

常用设置:

max, K = 2, S = 2

max, K = 3, S = 2 (AlexNet)

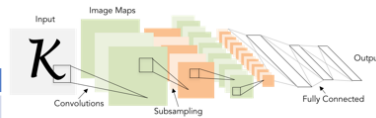
mean, avg, max

- 经典模型

- 结构

举例: LeNet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv (C _{out} =20, K=5, P=2, S=1)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool(K=2, S=2)	20 x 14 x 14	
Conv (C _{out} =50, K=5, P=2, S=1)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool(K=2, S=2)	50 x 7 x 7	
Flatten	2450	
Linear (2450 -> 500)	500	2450 x 500
ReLU	500	
Linear (500 -> 10)	10	500 x 10



经过整个网络:

空域大小**递减**
(通过池化和带步长的卷积)

通道数**递增**
(整个“体积”不变!)

特点

正则化

不同方法 (L1, L2, L1+L2, BN, Dropout)

添加正则项

简单示例

L2 正则化:

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

L1 正则化:

$$R(W) = \sum_k \sum_l |W_{k,l}|$$

结合 (L1 + L2):

$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

更复杂:

Dropout

Batch归一化

正则项目的:

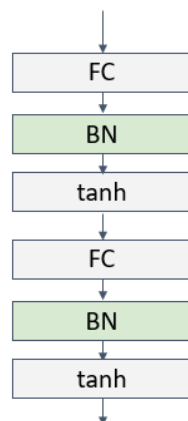
- 表达模型选择的倾向性除了“最小化训练错误”
- 避免过拟合: 倾向简单泛化性好的模型
- 提高优化效率通过增加弯曲

泛化 是指模型很好地拟合以前未见过的新数据 (从用于创建该模型的同分布中抽取) 的能力。

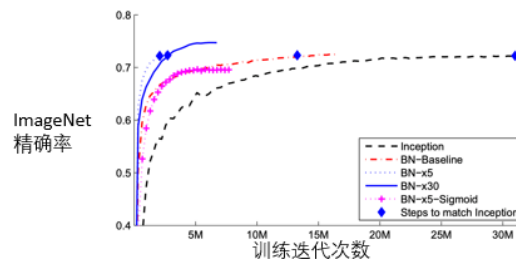
Dropout

向前传递过程中, 随机设置一些神经元的值为0 Dropout比例为超参数, 通常设置为0.5

归一化



- 使得深度网络更容易训练!
- 允许高的学习速率, 快速收敛
- 网络初始化更鲁棒
- 类似于训练过程中的正则化
- 测试过程中没有开销: 与 conv 融为一体!



归一化一层的输出, 使其具有零均值和单位方差-->帮助减少内部变量变化, 改进优化。通常插在全连接层或卷积层之后, 非线性层之前。

第二篇: 深度学习 (30%)

图像分类

- 定义

- 图像分类是根据图像的语义信息对不同类别图像进行区分，核心是从给定的分类集合中给图像分配一个标签的任务
- 图像分类任务目的是判断图像中包含物体的类别，如果期望判别多种物体则称为多目标分类。需要注意的是，基本的图像分类任务并不要求给出物体所在位置，也不需要判断含有物体的数量。

- 步骤：提取特征+分类

- AlexNet网络增大，同时解决过拟合，并且利用多GPU加速计算
- VGG通过增加网络深度，提取更准确的特征，提高分类结果
- GoogLeNet通过增加网络宽度，提取互补的特征，提高分类结果。
- ResNet通过残差块，把浅层特征传递深层，提高分类结果。

- 目标检测

- 定义

- 找出图像中所有感兴趣的目标（物体），确定它们的位置和大小。

- 步骤

- 选出目标，提取特征，分类。
- 两阶段目标检测方法：先生成候选框，后识别框内的物体：R-CNN、SPP-net、FastCNN、Faster CNN。
- 单阶段目标检测方法：没有中间的区域检测，直接冲图片重获得预测结果：YOLO、SSD

- 检测效果评估

目标检测子评估：
均值平均精确率Mean
Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 IoU > 0.5, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点
 2. 平均精确率(AP) = PR曲线下的面积
3. 均值平均精确率(mAP) = 平均各个类别的AP
4. 对于 "COCO mAP": 在不同的IoU阈值(0.5, 0.55, 0.6, ..., 0.95)下分别计算 mAP@thresh 并取平均

mAP@0.5 = 0.77
mAP@0.55 = 0.71
mAP@0.60 = 0.65
...
mAP@0.95 = 0.2

COCO mAP = 0.4

均值平均精确率 (Mean Average Precision) mAP

- 语义分割

- 图像分割类别与定义（此处列全部）

- 普通分割

- 如前景与后景分割开，狗的区域与猫的区域与背景分割开。

- 语义分割

- 在普通分割的基础上，分类出每一块区域的语义（即这块区域是什么物体）。

- 语义分割其目标是对图像中的每一个像素进行分类，自动从图像中分割出对象区域，并识别其中的内容
- 如把画面中的所有物体都指出它们各自的类别。
- 实例分割
 - 在语义分割的基础上，给每个物体编号。如这个是该画面中的狗A，那个是画面中的狗B。

• 步骤

- 提取像素特征，相同域的特征圈出来，从原图像中分割出来

• 第三篇：立体视觉（20%）

• 图像变换（先修）二维空间三位空间变换模型

• 旋转

$$\begin{array}{l}
 \mathbf{X} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{bmatrix} \leftarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \beta & -\sin \beta \\ \sin \beta & \cos \beta \end{bmatrix} \\
 \mathbf{Y} \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \leftarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \beta & -\sin \beta \\ \sin \beta & \cos \beta \end{bmatrix} \\
 \mathbf{Z} \begin{bmatrix} \cos \beta & -\sin \beta & 0 \\ \sin \beta & \cos \beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \leftarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \beta & -\sin \beta \\ \sin \beta & \cos \beta \end{bmatrix}
 \end{array}$$

• 图像拼接与融合（大作业涉及的内容）

• 步骤

- 特征点提取和匹配
- 计算变换矩阵进行图像配准
- 图像拷贝
- 图像融合

• 图像变形（内有ransac）

• 仿射变换：

• 仿射矩阵

仿射变换：

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

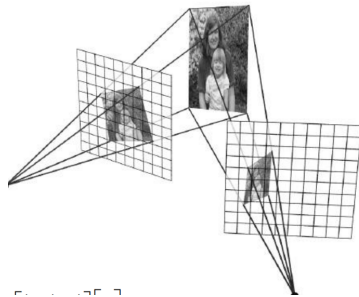
- 保持直线、平行
- 透射变换：

- 投射矩阵

投射变换:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- 不保持平行, 保持直线
- 单应变换矩阵
 - 目的
 - 将相机的图像投射到同一平面



- 若知道相应的相机参数:

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = M_{3 \times 4} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = M'_{3 \times 4} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = M'_{3 \times 4} * inv(M_{3 \times 4}) \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = N_{3 \times 3} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \quad \begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$\mathbf{p}' = \mathbf{H} \mathbf{p}$

- 求解 (不知道相机参数的情况下):

- 公式

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- 在齐次坐标中, 假设一点 $p(x_i, y_i, 1)$ 经过 H 矩阵的变换变为 $p'(x'_i, y'_i, 1)$, 即 $p' = H * p$, 通常, 对于透视变换, H 矩阵有 8 个自由度, 这样至少需要 4 对特征点对求解。4 个特征点对可以建立 8 个方程
- 有多个特征点对 -> 转化为优化问题 -> ransac

- Ransac

- [ransac算法图像匹配 RANSAC算法 weixin 39947396的博客-CSDN博客](#) (案例)

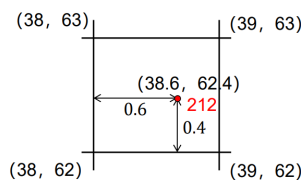
RANSAC的基本思想和算法流程如下：

1. 随机采样K个点，K是求解模型参数的最少点个数；
2. 使用K个点估计模型参数；
3. 计算剩余点到估计模型的距离，距离小于阈值则为内点，统计内点的数目；
4. 重复步骤1~3，重复次数M且保留数目最多的内点；
5. 使用所有的内点重新估计模型。

1. 随机选择一组种子点作为变换估计的基础（例如，一组匹配点）
2. 由这组种子点计算出映射矩阵
3. 按照这个映射矩阵计算出不符合这个矩阵的离群点
4. 如果非离群点的数量足够大，则重新计算所有非离群点的最小二乘映射矩阵
5. 保留具有最多非离群点的变换矩阵

- 图像插值

- 最近邻
- 双线性插值



将该点灰度值按权重分配给附近几个整像素位置

最近邻: $f(39,62) = 212$

双线性: $f(38,62) = 212 * (1-0.6) * (1-0.4)$

$$f(39,62) = 212 * 0.6 * (1-0.4)$$

$$f(39,63) = 212 * 0.6 * 0.4$$

$$f(38,63) = 212 * (1-0.6) * 0.4$$

- 应用场景

- 前向映射 $(x,y) \rightarrow x',y'$ ，上图例即为这，像素值分散开来
- 反向映射（与前向相反）

- 图像配准

- 分类
 - 基于模板的匹配
 - 基于特征的匹配
 - ransac见上面



- 提取特征点
- 计算假定匹配
- RANSAC循环：
 1. 提取几组假定匹配点对，计算变换矩阵
 2. 根据变换矩阵找到离群点和非离群点
 3. 找到最多非离群点，计算平均变换矩阵
- 其他匹配
 - 116-128
- 未看
 - 全景拼接 73-92
 - 图像融合方法（不考）
- 运动与光流
 - 背景
 - 运动估计
 - 概念
 - 运动估计研究的是视频序列图像中投影坐标在像平面上的变化，获取运动参数,但是投影会造成信息丢失（不可逆），导致估计误差。
 - 应用
 - 视觉运动可能引起图像模糊、相机不稳定、抖动。量出运动，加以补偿
 - 跟踪物体，分析运动轨迹
 - 视觉运动可以揭示空间布局——运动视差
 - 方法
 - 参数模型：物体运动被建模为某种映射形式。
 - 全局 --- 摄像机移动、单个运动物体
 - 区域 --- 多个运动物体（运动区域分割）
 - 非参数模型：对运动场施加平滑约束条件。
 - 光流、像素、随机场 --- 估计每个像素运动矢量
 - 块匹配 --- 分割成小的规则块，估计每个块的运动矢量
 - 网格 --- 分割成不重叠的多边形单元，可克服块失真
- 光流方程（基于光流的运动估计）
 - 光流是什么
 - 观测的或表现的二维运动。

- 不仅可以由物体运动引起，还可以由摄像机运动或照明条件变化引起。注:不等于真实的二维运动。

怎么推导?

- 假设恒定亮度

恒定亮度假设 → 光流方程

$$\psi(x + d_x, y + d_y, t + d_t) = \psi(x, y, t)$$

又运用泰勒展开式，当 d_x, d_y, d_t 很小时，

$$\psi(x + d_x, y + d_y, t + d_t) = \psi(x, y, t) + \frac{\partial \psi}{\partial x} d_x + \frac{\partial \psi}{\partial y} d_y + \frac{\partial \psi}{\partial t} d_t$$

$$\frac{\partial \psi}{\partial x} d_x + \frac{\partial \psi}{\partial y} d_y + \frac{\partial \psi}{\partial t} d_t = 0 \Leftrightarrow \frac{\partial \psi}{\partial x} v_x + \frac{\partial \psi}{\partial y} v_y + \frac{\partial \psi}{\partial t} = 0 \Leftrightarrow \nabla \psi^T \mathbf{v} + \frac{\partial \psi}{\partial t} = 0$$

其中 $\nabla \psi = \left[\frac{\partial \psi}{\partial x}, \frac{\partial \psi}{\partial y} \right]^T$ 为空间梯度向量， $\mathbf{v} = (v_x, v_y)$ 为光流场。

光流分解

- 孔径问题

孔径问题：光流方程只确定了法向的投影 v_n ，在切线的投影是不确定的，任何 v_t 的值都满足光流方程。

一个方程有两未知数——

要施加附加条件：最通常的约束是流矢量在空间平滑变化。使我们可以利用 $\mathbf{x}$ 周围一个小的邻域亮度变化去估计 $\mathbf{x}$ 处的运动。

运动估计仅在亮度变化区域可靠。

帧差方程（基于像素的运动估计）

- 是什么

- 像素 $\mathbf{x}$ 从 t_1 时刻到 $t_1 + dt$ 时刻的位移帧差（视频流的上下帧对应像素值做差）

怎么推导

- 像素 $\mathbf{x}$ 从 t_1 时刻到 $t_1 + d_t$ 时刻的位移帧差为：

$$e(\mathbf{x}, \mathbf{d}) = \psi(\mathbf{x} + \mathbf{d}, t_1 + d_t) - \psi(\mathbf{x}, t_1)$$

$\mathbf{d} = [d_x, d_y]^T$ 为 $\mathbf{x}$ 处的位移矢量

$$\psi(\mathbf{x} + \mathbf{d}, t_1 + d_t) = \psi(\mathbf{x}, t_1) + \frac{\partial \psi}{\partial x} d_x + \frac{\partial \psi}{\partial y} d_y + \frac{\partial \psi}{\partial t} d_t$$

$$e(\mathbf{x}, \mathbf{d}) = \frac{\partial \psi}{\partial x} d_x + \frac{\partial \psi}{\partial y} d_y + \frac{\partial \psi}{\partial t} d_t$$

由亮度守恒假设 $e(\mathbf{x}, \mathbf{d}) = 0$ ，即： $\frac{\partial \psi}{\partial x} d_x + \frac{\partial \psi}{\partial y} d_y + \frac{\partial \psi}{\partial t} d_t = 0$

两边同时除以 d_t ，可得： $\frac{\partial \psi}{\partial x} v_x + \frac{\partial \psi}{\partial y} v_y + \frac{\partial \psi}{\partial t} = 0$ （光流方程）

未看

- 其他

基于块的运动估计(光流法)

-

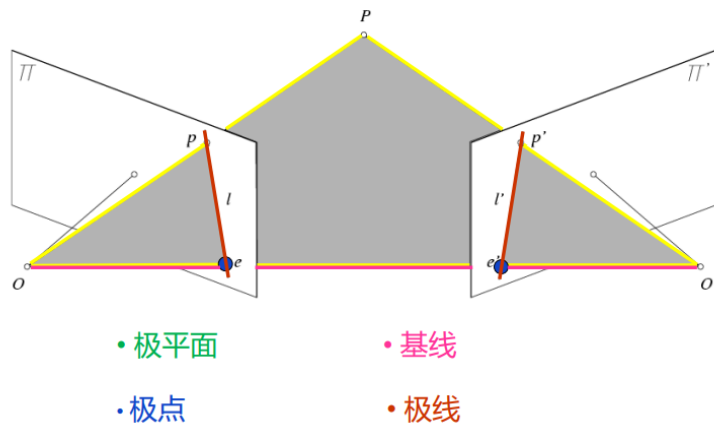
- 将图像分成一个个规则的图像块，对每个块进行运动估计。
- 广泛应用于各种视频压缩编码标准中。
- 块运动模型
 - 块平移模型
 - 假定每个块只做平移运动
 - 优点：实现简单
 - 缺点：1、不能表征旋转、缩放和局部变形；2、物体边界和块边界通常不一致，导致块失真；3、同一块可能包含多个运动物体。
 - 可变形块模型
 - 可以对物体的旋转、缩放、变形建模
 - 三种模型：投影运动、仿射运动、双线性运动
- 方法：相位相关法、块匹配法

• 立体视觉

• 背景

- 图像配准：稠密、稀疏
- 极线几何意义：将二维的匹配搜索限制在一维的搜索

• 极线几何（必考下图）



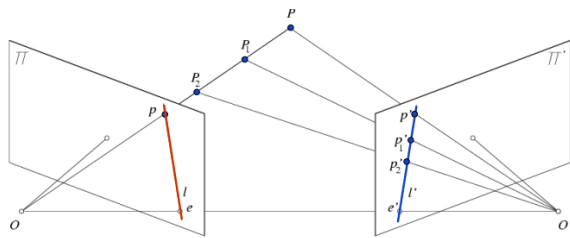
• 极线、极点、基线、极平面

• 关系

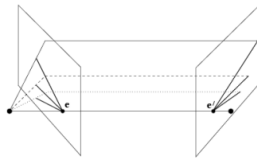
- **基线**: 两个光心的连线
- **极点**: 基线和成像平面交点
- **极平面**: 基线和物体点形成的平面
- **极线**: 极平面与成像平面的交线
- 所有极线都过极点。
- 物体改变，极平面会变，极线会变，但基线和极点不变。

• 变化

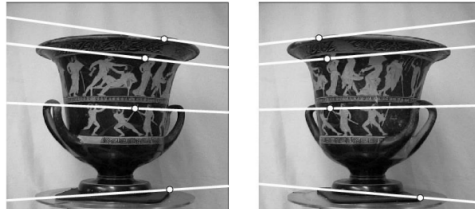
- 关注点在



- 左相机中点 p 在右相机的匹配点必然在极线 l' 上。
- 右相机中点 p' 在左相机的匹配点必然在极线 l 上。



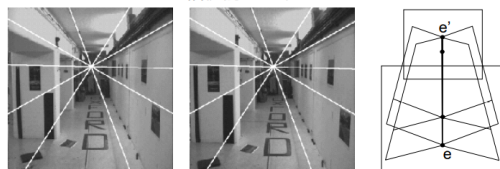
随着物体变化，极线绕着基线旋转



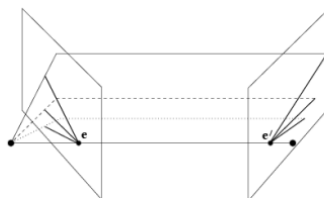
物体平行X轴移动



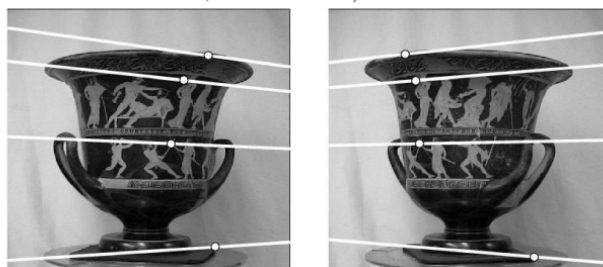
相机前向运动



- 两相机极点在同一位置
- 所有像素点会以关注点为圆心发散



随着物体变化，极线绕着基线旋转

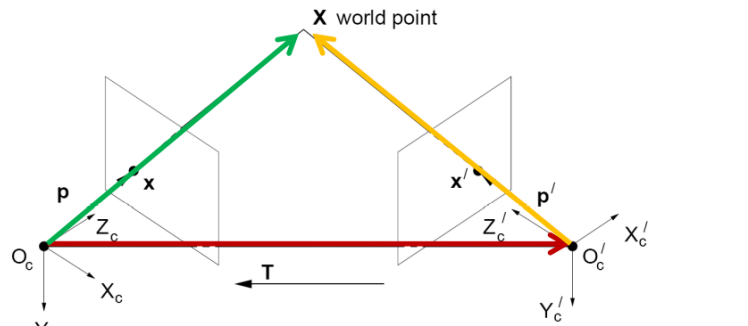


• 约束方程

- Essential本征矩阵：两个像点之间的物理坐标系关系

- 参考链接: [\(50条消息\) 立体视觉 \(Stereo Vision\) -本征矩阵 \(essential matrix\) 和基本矩阵\(fundamental matrix\)_吃瓜群众110的博客-CSDN博客](#)

- 推导



$$\boxed{\mathbf{X}'} = \boxed{\mathbf{R}\mathbf{X}} + \boxed{\mathbf{T}}$$

$$\mathbf{T} \times \mathbf{X}' = \mathbf{T} \times \mathbf{R}\mathbf{X}$$

$$\mathbf{X}' \cdot (\mathbf{T} \times \mathbf{X}') = \mathbf{X}' \cdot (\mathbf{T} \times \mathbf{R}\mathbf{X}) = 0$$

垂直极平面

Essential 矩阵

$$\mathbf{X}' \cdot (\mathbf{T} \times \mathbf{R}\mathbf{X}) = 0$$

$$\mathbf{X}' \cdot (\mathbf{T}_x \mathbf{R}\mathbf{X}) = 0$$

Let $\mathbf{E} = \mathbf{T}_x \mathbf{R}$

$$\mathbf{X}'^T \mathbf{E} \mathbf{X} = 0$$

点 $\mathbf{p}$ 和 $\mathbf{p}'$ 过光心的射线在 $\mathbf{X}$ 和 $\mathbf{X}'$ 射线上, 因此有:

$$\mathbf{p}'^T \mathbf{E} \mathbf{p} = 0$$

- Fundamental基本矩阵:

- 推导

$$q_r = M_r p_r \quad q_l = M_l p_l \quad M_r M_l \text{ 是相机内参矩阵}$$

$$(M_r^{-1} q_r)^T E M_l^{-1} q_l = 0 \quad \Rightarrow \quad q_r^T M_r^{-T} E M_l^{-1} q_l = 0$$

$$F = M_r^{-T} E M_l^{-1} \quad \text{则:} \quad q_r^T F q_l = 0 \quad \text{基本矩阵}$$

- 运动恢复 不考

- 致谢

- 第一代作者: 罗心雨、王涵涵、卢军汀
- 第二代作者: 项裕顺、肖馨艺、白顺文、翁宇哲、楚逸飞